

Introduction to CS111



CS111 Computer Programming

Department of Computer Science
Wellesley College

Am I in the right class?

- **CS111** introduces the fundamentals of programming and problem solving techniques using Python.
- More advanced concepts are taught in **CS230: Data Structures**.
- **CS110: Computer Science and the Internet**, teaches web design.
- **CS112: Computation for the Sciences**, teaches programming skills using MATLAB. [Spring 2016]
- **CS117: Inventing Mobile Apps**, teaches CS in the context of making apps for Android smartphones. [Fall 2015]
- **CS114: The Socio-Techno Web**, teaches the structure of the social web. [Not offered this year]



Getting Started 1-2

Lectures and Labs

- Lectures (TF 9:50—11:00am or 11:10am-12:20pm or 1:30-2:40pm) introduce and discuss material.
- Labs provide hands on experience with the ideas presented in lecture. There are seven lab sections:

Wed L01 8:30 - 10:20am (Sohie Lee, SCI257)
Wed L02 10:30am - 12:20pm (Sohie Lee, SCI257)
Wed L03 2:15 - 4:05pm (Sohie Lee, SCI257)
Thu L04 8:30 - 10:20am (Jean Herbst, 160A)
Thu L05 10:30am - 12:20pm (Jean Herbst, 160A)
Thu L06 8:30 - 10:20am (Sohie Lee, E101)
Thu L07 1:30 - 3:20pm (Sohie Lee, E101)



- If you cannot get into the section you want, register for another and use the [cs111-fall15 Google Group](#) to arrange a swap.

*This does **not** satisfy the Wellesley laboratory requirement.

Getting Started 1-3

You needn't go to the bookstore

- All course materials may be found at <http://cs.wellesley.edu/~cs111>
- The CS file server, cs.wellesley.edu, is used to download and upload programming assignments.*
- Textbook - you don't have to buy one. We will rely on resources available on the web, particularly Allen Downey's [Think Python](#).
- Many intro Python programming books can be found in other places:
 - Bookshelves in SCI 257
 - Bookshelves in SCI 160A
 - Science Center library

*Assignment 0, due by 7pm today, gets you an account.

Lab 1 shows you how to use it.

Assignment 1 is due next Tuesday

Getting Started 1-4

Help

- Q&A and announcements will be posted on the [cs111-fall15 Google group](#). Read this daily and/or have posts forwarded to your email.
- Lecturers, lab instructor, and TAs have office/drop-in hours that are posted on the course web site.
- We'd like to get to know you. Come see us, even if you don't have a question.
- PLTC tutors are available to those who want them (at no charge to you).



Getting Started 1-5

Most importantly, talk to each other. But...

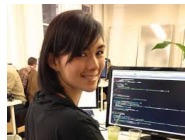
- All discussion should be in a high level language (drawings, English). Do **not** share code: it is a violation of the Honor Code.
- One exception:** you can code together with a partner on designated **pair programming** assignments (more on this later in semester).
- Do not consult materials from previous semesters. This also violates the Honor Code.
- When in doubt, ask!



Getting Started 1-6

Assignments: 3 kinds of problems

Solo (Embrace the Challenge): you must do these problems completely on your own, without help from fellow students or the 111 staff.



Guided (Build up the Strength): these are individual problems on which you can get help from the 111 staff.



Partner (Teamwork/Pair Programming): these are problems on which you work with a partner and can get help from 111 staff. Your pair turns in a single solution.

Getting Started 1-7

What is Computer Science?

- It's not really about computers.
- It's not really a science.
- It's about **imperative ("how to") knowledge** as opposed to declarative ("what is") knowledge.
- Imperative knowledge is expressed via **algorithms = computational recipes**.
- "A computer language ... is a novel formal medium for expressing ideas about methodology, not just a way to get a computer to perform operations. Programs are written for people to read, and only incidentally for machines to execute"
-- Harold Abelson and Gerald J. Sussman

Getting Started 1-8

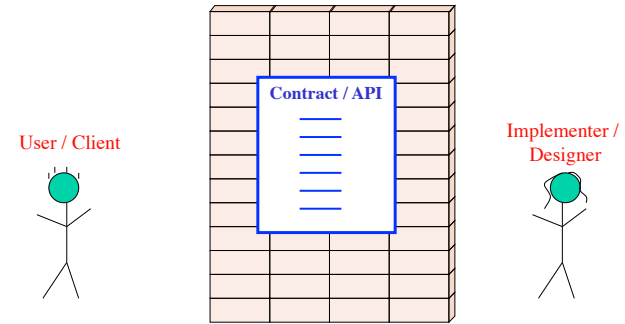
Four big ideas

- Four important concepts are at the core of this course:
 1. Abstraction;
 2. Modularity;
 3. Divide, Conquer and Glue;
 4. Models
- These interrelated ideas are important in almost every discipline, but they're at the core of CS.
- We will illustrate these ideas in several ways.
- Our goal is to help you think about problem solving in new ways.



Getting Started 1-9

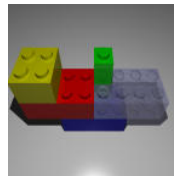
Big idea number 1: Abstraction



Getting Started 1-10

Big idea number 2: Modularity

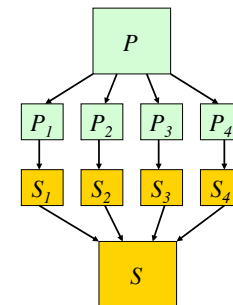
- Large systems are built from components called **modules**.
- The interfaces between modules are designed so they can be put together in a mix-and-match way.
- In computer programming, the goal is to design packages for maximum reusability.



Getting Started 1-11

Big idea number 3: Divide, conquer & glue

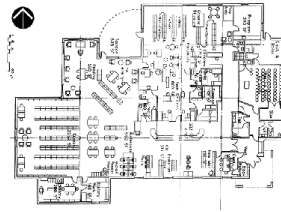
- Divide**
problem P into subproblems.
- Conquer**
each of the subproblems.
- Glue (combine)**
the solutions to the subproblems into a solution S for P .



Getting Started 1-12

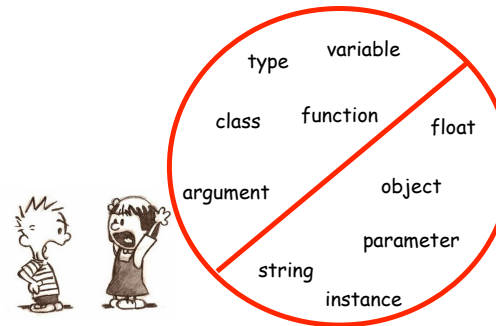
Big idea number 4: Models

- Need simple models to understand complex artifacts and behaviors.



Getting Started 1-13

Unlearn what you have learned

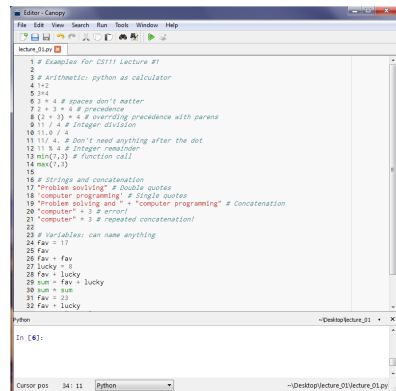


I don't think that word means what you think it means

Getting Started 1-14



helps you edit and execute
Python programs



Getting Started 1-15

Introduction to Python: Python as calculator

Input expressions
In [...]

1+2		3
3*4		12
3 * 4	# Spaces don't matter	12
3.5 * 2.0	# Floating point (decimal) operations	7.0
2 + 3 * 4	# Precedence	14
(2 + 3) * 4	# Overriding precedence with parentheses	20
11 / 4	# Integer division	2
11.0 / 4.0	# Floating point (decimal) division	2.75
11 / 4.0	# Floating point (decimal) division	2.75
11 % 4	# Integer remainder	3

Output Values
Out [...]

Getting Started 1-16

Strings and concatenation

A string is just a sequence of characters that we write between a pair of double quotes or a pair of single quotes.

In [...]	Out [...]
"CS111" # Double quotes	'CS111'
'rocks!' # Single quotes	'rocks!'
'CS111 ' + 'rocks!' # Concatenation	'CS111 rocks!'
'111' + 5 # Type error	Error!
'111' + '5' # Concatenation	'1115'
111 + 5	116
'111' * 5 # Repeated concatenation	'1111111111111111'

Getting Started 1-17

Variables

A variable is a **box** containing a value that a programmer names or changes with assignment statement, using =. Variables can name any value.

Note: The symbol = is pronounced "gets" not "equals"!

In [...]	Out [...]
fav = 17	# assignment statement has no output
fav	17
fav + fav	34
lucky = 8	
fav + lucky	25
a_sum = fav + lucky	
a_sum * a_sum	625
fav = 12	
fav = fav - lucky	
name = 'CS111'	
name * fav	'CS111CS111CS111CS111'

Getting Started 1-18

Built-in functions: max and min

Python has many **built-in functions**. Their names are shown in a green color in Canopy. Variable names are black.

In [...]	Out [...]
min(7,3)	3
max(7,3)	7
min(7,3,2,9) # can take any num. of arguments	2
smallest = min(-5, 2)	# smallest gets -5
largest = max(-3, -10)	# largest gets -3
max(smallest, largest, -1)	-1

The inputs to a function are called its **arguments** and the function is said to be **called** on its arguments. In Python, the arguments in a function call are delimited by parentheses and separated by commas.

Getting Started 1-19

Built-in functions: type

Each Python value has a type. It can be queried with the built-in **type** function. Types are special kinds of values (not strings!)

In [...]	Out [...]
type(111)	int
type(4.0)	float
type('CS111')	str
type('111')	str
type(7/4)	int
type(7.0/4.0)	float
type(7.0/4)	float
type(max(7,3))	int
x = min(7,3)	
type(x)	int
x = 'CS111' + 'rocks!'	
type(x)	str
type(type(111))	type

Getting Started 1-20

Built-in functions: `len`

When applied to a string, the built-in `len` function returns the number of characters in the string.

```
In [...]
len('CS111')
len('CS111 rocks!')
len('com' + 'puter')
course = 'computer programming'
len(course)
len(111)
```

```
Out [...]
5
12
8
20
# Error
```

Getting Started 1-21

Built-in functions: `str`

The `str` built-in function returns a string representation of its argument

```
In [...]
str('CS111')
str(17)
str(4.0)
'CS' + 111
'CS' + str(111)
len(str(111))
len(str(min(17,3)))
len_of_name = len('CS' + str(max(110,111)))
str(len_of_name)
```

```
Out [...]
'CS111'
'17'
'4.0'
# Error!
'CS111'
3
1
5
```

Getting Started

1-22

Built-in functions: `int`

When given a string that's a sequence of digits, optionally preceded by +/-, `int` returns the corresponding integer.

When given a floating point number, `int` truncates it toward zero.

When given an integer, `int` returns that integer.

```
In [...]
int('42')
int('-273')
123 + '42'
123 + int('42')
int('3.141')
int('five')
int(3.141)
int(98.6)
int(-2.978)
int(42)
```

```
Out [...]
42
-273
# Error!
165
# Error!
# Error!
3
98
-2
42
```

Getting Started 1-23

Built-in functions: `float`

When given a string that's a sequence of digits, optionally preceded by +/-, and optionally including one decimal point, `float` returns the corresponding floating point number.

When given an integer, `float` converts it to floating point

When given a floating point number, `float` returns that number.

```
In [...]
float('3.141')
float('-273.15')
float('3')
float('3.1.4')
float('pi')
float(42)
float(98.6)
```

```
Out [...]
3.141
-273.15
3.0
# Error!
# Error!
42.0
98
```

Getting Started

1-24

Vagaries of floating point numbers

In computer languages, floating point numbers (numbers with decimal points) don't always behave like you might expect from mathematics. This is a consequence of their fixed-sized internal representations, which permit only approximations in many cases.

In [...]	Out [...]
2.1 - 2.0	0.10000000000000009
2.2 - 2.0	0.20000000000000018
2.3 - 2.0	0.2999999999999998
1.3 - 1.0	0.30000000000000004
100.3 - 100.0	0.2999999999999716
10.0/3.0	3.3333333333333335
1.414*(3.14159/1.414)	3.1415900000000003

Getting Started 1-25

Built-in functions: `round`

When given a single numeric argument, `round` returns a floating point version of the integer it's closest to.

When given two arguments (a numeric argument and an integer number of decimal places), `round` returns the result of rounding the first argument to the number of places specified by the second.

In [...]	Out [...]
<code>round(3.14156)</code>	3.0
<code>round(98.6)</code>	99.0
<code>round(-98.6)</code>	-99.0
<code>round(3.5)</code>	4.0
<code>round(4.5)</code>	5.0
<code>round(2.718, 2)</code>	2.72
<code>round(2.718, 1)</code>	2.7
<code>round(2.718, 0)</code>	3.0
<code>round(1.3 - 1.0, 1)</code> # compare prev. slide	0.3
<code>round(2.3 - 2.0, 1)</code> # compare prev. slide	0.3

Getting Started 1-26

Built-in functions: `print`

`print` displays a character-based representation of its argument(s) on the screen. It does not evaluate to a result value.

Input statements
In [...]

```
print(7)
print('CS111')
print('CS' + 111)
print('CS' + str(111))
print(len(str('CS111')) * min(17,3))
college = 'Wellesley'
print('I go to ' + college)
dollars = 10
print('The movie costs '
      + str(dollars) + ' dollars.')
print(1+2, 6*7, 'foo' + 'bar')
```

Characters displayed in
console (*not* the output
value of the statement!)

```
7
CS111
# Error!
CS111
15

I go to Wellesley

The movie costs 10
dollars.
3 42 foofoo
```

* The final example shows an idiomatic use of `print` with commas to display multiple values on the same line

Getting Started 1-27

Expression values vs. `print`

```
In [1]: max(10,20)
Out[1]: 20

In [2]: 10 + 20
Out[2]: 30

In [3]: message = "Welcome to CS 111"

In [4]: message
Out[4]: 'Welcome to CS 111'

In [5]: print(message)
Welcome to CS 111

In [6]: print(max(10,20))
20

In [7]: print(10 + 20)
30
```

Notice the field `Out[]` when the input is a function call, expression, or variable.

The function `print` doesn't output a value, it only displays the result on the screen.

More built-in functions: `raw_input`

`raw_input` displays its argument on the screen and waits for the user to input text, followed by Enter/Return. It returns the string entered.

```
In [1]: raw_input('Enter your name: ')
Enter your name: Phil E. Buster
Out [1]: 'Phil E. Buster'
```

----- Brown text is prompt.
----- Magenta text is entered by user.

```
In [2]: age = raw_input('Enter your age: ')
Enter a number: 19
```

----- Variable assignment. No output.

```
In [3]: age
Out [3]: '19'
```

----- Tried to add a string and a float

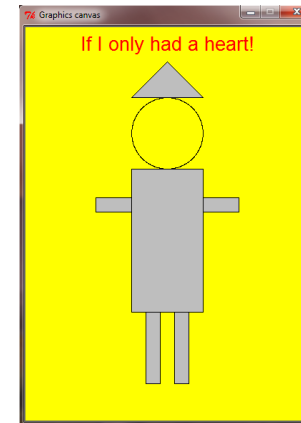
```
In [4]: age + 4.0
Out [4]: Error!
```

----- Example of "nesting" two functions

```
In [5]: age = float(raw_input('Enter your age: '))
Enter a number: 19
In [6]: age + 4.0
Out [6]: 23.0
```

Getting Started 1-29

Example with `cs1graphics`



Getting Started 1-30